

LCC TOPICS

Using Push Buttons and Toggles

Revised: 2/14/2025 Created: 11/14/21

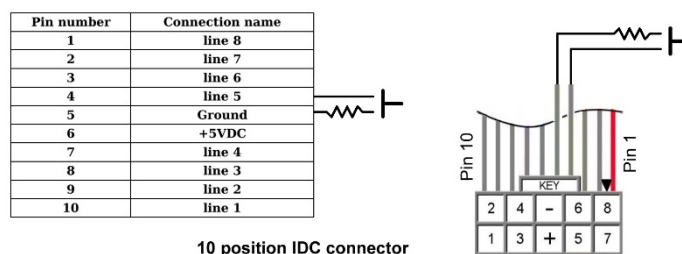
There are many circumstances using LCC in model railroading when one wants to Produce two EventIDs to control something that has two possible states. Examples include:

- Turnout position.
- Crossing gate position.
- Relay on/off for room lights or layout lights.
- LED on/off.
- Veto On and Veto Off states of a Node Line.

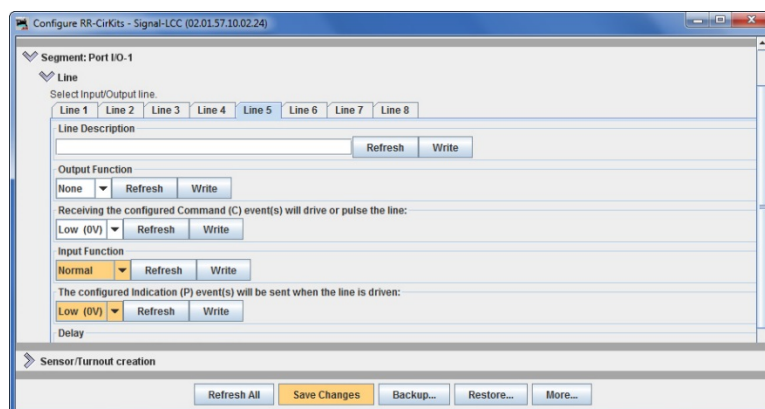
This document presents several ways of Producing a pair of EventIDs using RR-CirKits LCC Nodes.

The examples below are based on the RR-CirKits SignalLCC with firmware revision C6a and TowerLCC with firmware revision C7a using the JMRI 4.24 CDI tool on a Windows 7 Pro PC.

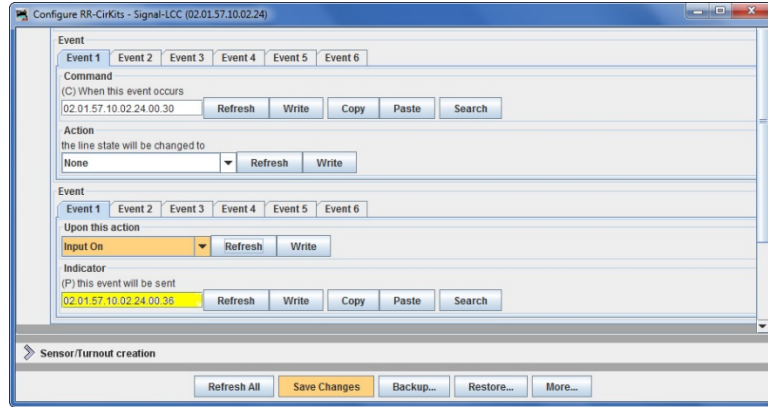
- USE PUSH BUTTONS.** These can be attached to a BOD-4-CP (See LCC Topic titled *Understanding BOD-4-CP*) or attached directly between pin 1, 2, 3, 4, 7, 8, 9, or 10 and pin 5 (Ground, common) on the Node I/O connector. If attached directly to a Node Line being used for both input and output, a 1 kΩ resistor must be in series with the push button. If used only for input, the resistor is not necessary. The figure below shows a normally open push button connected to Node Line 5. Note that the I/O connector pin number is not the same as the Node Line number found in the CDI.



- Using two push buttons to generate the two EventIDs. These can be attached to the same Node or to different Nodes. Configure the Node Line each button is attached to as a Normal Input:

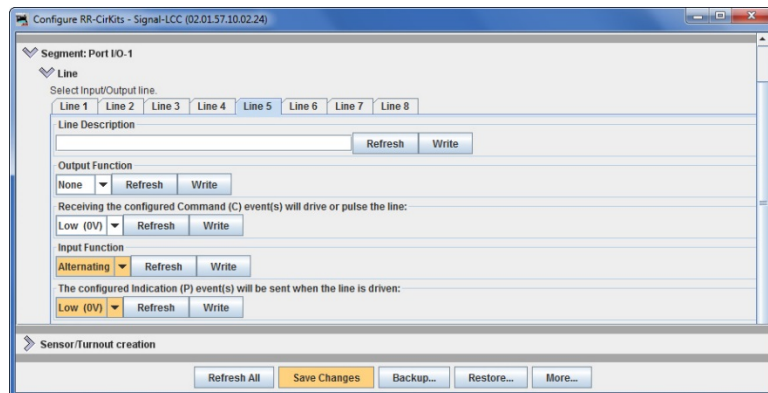


Then configure an Event for this Line to Produce an EventID when the button is pushed. In this case, the button is normally open so the input is High and goes Low when the button is pushed. We configured the Line to be On when the input is Low (0V), so Event 1 has been Configured with 'Upon this action' = "Input On" to send the EventID highlighted in yellow. This EventID can be used to cause one of the two Consumer actions. Alternatively, an EventID taken from the desired Consumer can be copied and pasted here (See LCC Topic titled *Understanding EventIDs*).

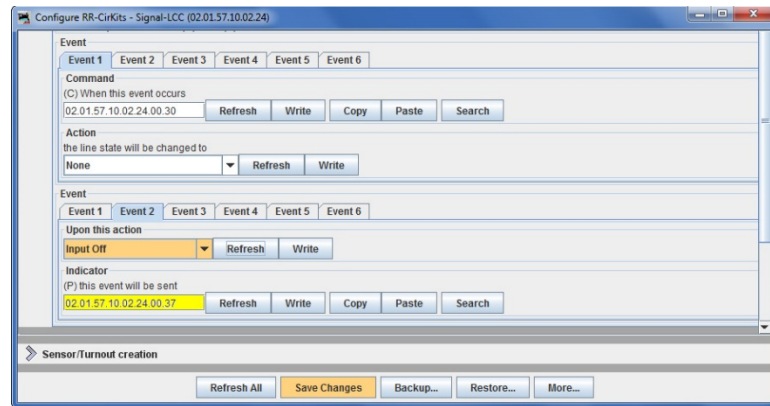


To Produce the second EventID, just configure the other push button in the same manner as the first, but with a different EventID. Again, you can either copy and paste the EventID from this Producer into the second Consumer action, or the other way around.

2. [Using a single button](#) to generate the two EventIDs. If we configure a push button as an Alternating Input, the node's line state will toggle between On and Off with each push of the button.

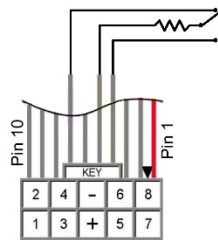


We can then Produce one EventID when the input is On and the other when the input is Off. The Event 1 configuration for the On state is the same as in 1 above. Event 2 for the Off state just involves changing 'Upon this action' to "Input Off".

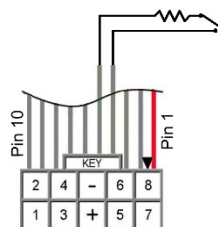


Note the similarity and difference between the two button and the one button approach. Both Produce two different EventIDs. In the two button case, they come from pushing different buttons which only requires a single event for each button. Pushing a particular button puts the Consumer into a known state associated with the button. In the one button case, they come from the same button and use two events for that button. Pushing the button toggles the Consumer rather than putting it in a known state. In both cases though, each EventID is associated with a known state of the Consumer, so the two EventIDs can be used by other Consumers to track and react to the state of the target Consumer.

- B. **USE TOGGLE SWITCHES.** These can be attached to a BOD-4-CP or attached directly between pin 1, 2, 3, 4, 7, 8, 9, or 10 and Ground (common) on the Node I/O connector just like push buttons. Keep in mind that unlike a push button which is momentary, a toggle switch (or latching push button) is not momentary. This means that it may take two throws of a toggle to achieve the same result as a single push of a button.
1. **Use a SPDT toggle switch** as if it were two push buttons. Each pole of the switch will act as a push button that is held down. Just configure each Node Line the same way the two push buttons were configured. The figure below shows a Single Pole Double Throw toggle switch attached directly to Lines 4 & 5 of an LCC Node. In this case, you would configure Lines 4 & 5 as was done in A.1 above. As with pushbuttons, the resistor is only required if the line is used for both input and output.



2. **Use a SPST toggle switch** as if it were a single latching push button. Unlike A.2, however, the action of the toggle will provide the On/Off states of the input, so use 'Input Function' = "Normal". The figure below shows a Single Pole Single Throw toggle connected directly to Line 5. Configure two events for this Line as was done in A.2 above.



Note that B(1) and B(2) appear the same to the model railroader because both use just a single toggle switch.

VARIATIONS:

LCC provides multiple ways to achieve the same result, so you need to decide which approach works best for your style and the circumstance at hand. For instance,

- Instead of using the pressing of a push button to Produce an EventID, you can use the release of the button.
- Instead of using a normally open push button, you can use a normally closed push button. In this case, the quiescent state of the Node Line input will be Low (0V) and pressing the button will cause the input to go High (5V).
- Instead of using old style push buttons or toggle switches, you can use other devices such as Berrett Hill Touch Toggles or Touch Triggers. See for example, <https://youtu.be/vtFweVHYU8> . Warning – this video shows an older Node firmware version where the CDI is different with respect to configuring a Line as input, output, or both.
- If you are handy with JMRI tables, you can define a sensor or turnout and use a click on the “State” entry in the table to Produce the corresponding EventID. See LCC Topic titled *Understanding JMRI Tables & CDI Search* for how to use the CDI to create JMRI table entries.

PUSH BUTTONS ARE GENERALLY PREFERRED

It is not uncommon for a layout to have a global ‘set-to-default’ EventID to put things, such as turnout positions, into a known starting condition. Keep in mind LCC is all about events and state *changes*, not states. So even if the toggles are polled at startup, if something else sends a command that conflicts with the state of the toggle switch (say, throwing a turnout via another push button or via the PC or the set-to-default event), the toggle will be out of position. This can get things out of sync, particularly if some logic is using the toggle.

The issue really boils down to which is the master control. For a toggle, it’s physical position is meaningful. You should let it be the master for the state and have it ignore the set-to-default request. But that might create other issues.

Bottom line is try to avoid using toggle switches in an LCC system. You can make them work, but it may introduce a lot of situations that you need to consider, particularly if you want the toggle’s physical position to have some meaning.

STAYING IN-SYNC:

LCC is very flexible. Production of two EventIDs need not be restricted to one or two push buttons or one toggle switch. You could have two push buttons on the layout fascia and two other push buttons on a dispatcher panel to control one turnout. You just have to be careful that each pair of buttons Produce the same pair of EventIDs. You could also have a single button in one location and two buttons in the other, or a single button in one location and a toggle switch in the other, or any other combination. If using one or more single push buttons you don’t have to worry about the action of another button or switch causing things to get out-of-sync because a Node configured for an alternating input will monitor the EventIDs involved and keep the action of the alternating input in-sync.

GETTING OUT-OF-SYNC:

There are circumstances where multiple push buttons configured for alternating input can get out-of-sync when controlling a common Consumer.

- 1) The target consumer action is configured to respond to more than one EventID pair. If one of the buttons or some other producer using the common EventID pair is used, all the Producers using that pair will stay in sync. If the other EventID pair is used, the producers

using the common pair will get out-of-sync because the Consumer will respond but the producers will not receive a common EventID and will thus not be aware that the consumer has changed.

- 2) A Veto is applied to one or more of the Producers. The Veto status affects an event, it does not affect the alternating input action. If a button is pushed and the Producer event is vetoed, the input will toggle, but the EventID will not be sent. Thus, the Consumer will remain unchanged and the other buttons will still be in-sync, but the pressed button will be out-of-sync. This means it is possible that when the veto is removed, the button may have to be pressed twice instead of once.
- 3) A veto is applied to the Consumer. If an alternating input button is pressed, the appropriate EventID will be sent and the other buttons sharing the EventID pair will remain in-sync with one another. However, the Consumer will not react, so all the buttons will now be out-of-sync with the Consumer. When the veto is removed, the next button used may have to be pressed twice.